Shiv Nadar University - Chennai

Department of Computer Science & Engineering

25CS2741 Fundamentals of Internet of Things + Lab

Date: 27/10/2026 (Week 13) **Continuous Lab Evaluation-13** II Yr/ 3rd Sem B.Tech CSE(IoT)A&B

Allotted Marks: 10 (Hardware Implementation + Code Efficiency + Viva-Voce)
Tools: CubeMX + Keil MDK + **STM32 Nucleo Development Board** + **Debugger**
Custom Built ESP8266 API (MQTT) + STM32 HAL libraries are allowed

Sensor Interfacing (BME280) – (MQTT) – Node Analytics

BME280 Sensor Interfacing Using I^2C and (MQTT) using Custom Built ESP8266 API

As continuation of the previous exercise, Develop an STM32-based environmental sensing application to interface a BME280 sensor using the I^2C protocol and obtain compensated temperature, pressure, and humidity measurements top Node-Red For analytics using MQTT.
Use the included custom-made ESP8266 library for enabling MQTT

The required driver files (h & .c files) shall be sourced from the shared repository.

Step 1: Download and Arrange the Driver Files
Arrange the **two specific files** from the src and Inc folder:

- ESPLib.c
- ESPLib.h

Step 2: Include the Files in Keil uVision

1. Add the .c file to the workspace:
    - Expand your target folder and locate the group named Application/User/Core
    - Right-click the group folder → Select Add Existing Files to Group...
    - Browse to Core/Src, select bme280.c, and click Add
2. **Verify Include Paths (If using standard paths):**
    - Click the **Options for Target** magic wand icon - Go to the **C/C++** tab.
    - Under **Include Paths**, ensure your project's include folder (e.g., ..\Core\Inc) is present.

Step 3: Implement Interface Wrappers
Open your main.c file and construct your code using this framework:

1. Initialization -

```
/* USER CODE BEGIN 2 */
  // 1. Initialize the Wi-Fi module on USART2
  ESP8266_Init(StationMode, huart2);

  // 2. Connect to your local Wi-Fi network (Replace with your Router SSID and Password)
  ESP8266_wifiConnect("*********", "********");

  // 3. Connect to your MQTT Broker (Replace with your Broker's IP Address)
  // 1883 is the standard unencrypted MQTT port
  ESP8266_portConnect("TCP", "broker.emqx.io", "1883");

  // 4. Configure the MQTT client connection parameters
  MQTT_InitTypeDef_t mqttSetting;
  mqttSetting.keepAlive = 500;
```

2. Inside While(1)

Place these functions in main.c

```
// for Publishing Only
while (1)
 {
        MQTT_publishTopic("stm32/test/data", "hello", QoS0);
        HAL_Delay(1000);
 }
```

```
// For Subscribe Only

char mqttData[50];
char mqttTopic[10];

int main(void)
{
 ESP8266_Init(StationMode, huart2);
 ESP8266_wifiConnect("******","******");
 ESP8266_portConnect("TCP", "**********", "1883");

 MQTT_InitTypeDef_t mqttSetting;
 mqttSetting.keepAlive = 500;
 mqttSetting.connectFlag = CleanSession;
 mqttSetting.clientID = "MqttID";
 MQTT_connectBroker(&mqttSetting);
 MQTT_subscribeTopic("IOT", 0);
 HAL_UART_Receive_IT(&huart2 , &uartRxData , 1);
```

```
while (1)
 {
        MQTTDataHandler(mqttTopic, mqttData);
 }
}
void SysTick_Handler(void)
{
 /* USER CODE BEGIN SysTick_IRQn 0 */
 MQTTTimer();
 /* USER CODE END SysTick_IRQn 0 */
 HAL_IncTick();
 /* USER CODE BEGIN SysTick_IRQn 1 */

 /* USER CODE END SysTick_IRQn 1 */
}
```

Develop the application.

Using the Bosch BME280 API and its relevant header/source files:

- Include the required BME280 driver files.
- Declare the BME280 device structure.

STM32 should construct the JSON payload from the compensated BME280 readings.

For example:

{"status":"success","temperature":28.45,"humidity":62.30,"pressure":1008.72}

Next step — Node-RED

Use **Mqtt_in** Function in Node-Red Palette (under Network Category) to receive the Json format inside Node-Red to visualize the three parameters of the BME sensor.

For Reference :

https://www.emqx.com/en/blog/using-node-red-to-process-mqtt-data

